Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение высшего образования

«Алтайский государственный технический университет им. И. И. Ползунова»

Факультет информационных технологий Кафедра информатики, вычислительной техники и информационной безопасности

Отчет защищен с оценкой_____
Преподаватель С. В. Умбетов
«____»_____2026 г.

Отчёт по лабораторной работе №3 по дисциплине «Устройство и функционирование интеллектуальных информационных систем» «Введение в обработку текста (NLP Basics) на русском языке»

ЛР 09.02.08.21.003

Студент группы 1ИИС-41 Т.Е.Рябич
группа и.о., фамилия
Преподаватель доцент, к. т. н. С. В. Умбетов
должность, ученая степень и.о., фамилия

БАРНАУЛ 2026

Цель работы

Изучить базовые этапы предварительной обработки текстовой информации: токенизацию, фильтрацию шума с помощью регулярных выражений, удаление стоп-слов, лемматизацию на базе библиотек spaCy и NLTK, а также расчет статистической меры важности слов методом TF-IDF.

Задачи к работе

1. Реализовать глубокий анализ, фильтрацию и расширение базового списка стоп-слов.
2. Исследовать динамику изменения признакового пространства векторов при масштабировании текстового корпуса.
3. Модифицировать регулярные выражения под задачу сохранения цифровых метрик в тексте.

Задание принято:  **Рябич Т.Е.**

Подпись

Ход работы

Задание 1. Глубокий анализ и кастомизация стоп-слов

```
!pip install spacy
!python -m spacy download ru_core_news_sm

import re
import math
import spacy
import nltk
from nltk.corpus import stopwords

nltk.download('stopwords')
nlp = spacy.load("ru_core_news_sm")

corpus = [
    "Сегодня была отличная погода, и мы пошли гулять в красивый парк!",
    "Ужасная погода ... Весь день идет сильный дождь, прогулка отменяется.",
    "В парке мы видели красивых птиц. Погода шепчет, солнце светит ярко.",
    "Дождь не прекращается! Сильный ветер сорвал все планы на вечер."
]
```

Рисунок 1 — Инициализация рабочего окружения в Google Colab, загрузка русскоязычной языковой модели spaCy, стоп-слов библиотеки NLTK и определение исходного корпуса текстов.

```
russian_stopwords = set(stopwords.words('russian'))
print(len(russian_stopwords))
print(sorted(list(russian_stopwords)))

151
['а', 'без', 'более', 'больше', 'будет', 'будто', 'бы', 'был', 'была', 'были', 'было', 'быть', 'в', 'вам', 'вас', 'вдруг', 'ведь', 'во', 'вот', 'впрочем', 'все', 'всегда', 'всего', 'всех', 'всю'
```

Рисунок 2 — Вывод общего количества русских стоп-слов из библиотеки NLTK и полный список слов, отсортированный по алфавиту.

Классификация 5 выбранных стоп-слов из базового списка NLTK:

1. **Личное местоимение:** «мы»
2. **Предлог:** «в»
3. **Союз:** «и»
4. **Частица:** «не»
5. **Указательное слово:** «это»

Рисунок 3 — Выбор пяти слов из базового списка стоп-слов NLTK и их лингвистическая классификация по частям речи.

```
russian_stopwords = set(stopwords.words('russian'))
custom_stopwords = russian_stopwords.copy()
custom_stopwords.update(['сегодня', 'весь'])

def pipeline_preprocess(text, stop_words):
    text = text.lower()
    text = re.sub(r'[^а-яё\s]', '', text)
    doc = nlp(text)
    return [token.lemma_ for token in doc if token.lemma_ not in stop_words and token.lemma_.strip()]

doc1 = corpus[0]
tokens_before = pipeline_preprocess(doc1, russian_stopwords)
tokens_after = pipeline_preprocess(doc1, custom_stopwords)

print("До:", tokens_before)
print("После:", tokens_after)
```

```
До: ['сегодня', 'отличный', 'погода', 'пойти', 'гулять', 'красивый', 'парк']
После: ['отличный', 'погода', 'пойти', 'гулять', 'красивый', 'парк']
```

Рисунок 4 — Добавление двух контекстно-зависимых слов («сегодня», «весь») в базовый набор стоп-слов и сравнительный анализ изменения состава токенов для первого документа.

Задание 2. Исследование масштабируемости корпуса и динамики векторов

```
new_doc = "Сегодня сильный ветер принес новые тучи."
extended_corpus = corpus + [new_doc]

def get_vocabulary(processed_corp):
    return sorted(list(set([word for doc in processed_corp for word in doc])))

processed_old = [pipeline_preprocess(text, russian_stopwords) for text in corpus]
processed_new = [pipeline_preprocess(text, russian_stopwords) for text in extended_corpus]

vocab_old = get_vocabulary(processed_old)
vocab_new = get_vocabulary(processed_new)

print(len(vocab_old))
print(len(vocab_new))
print(set(vocab_new) - set(vocab_old))
print(len(vocab_old))
print(len(vocab_new))
```

```
26
29
{'туча', 'новый', 'принести'}
26
29
```

Рисунок 5 — Перезапуск пайплайна обработки текста, добавление нового пятого документа в корпус и фиксация изменения размеров уникального словаря.

Математическое обоснование весов TF-IDF:

Вес новых слов в пятом документе оказался выше, чем у повторяющихся слов, из-за их минимальной глобальной частоты во всем текстовом массиве. При расчете по формуле **IDF**:

$$IDF = \log\left(\frac{\text{Всего документов}}{\text{Документов со словом}}\right)$$

- **Уникальное новое слово** (например, *«туча»*): зафиксировано только в 1 документе из 5.
$$IDF = \log(5/1) \approx 1.61$$
- **Повторяющееся старое слово** (например, *«ветер»*): присутствует сразу в 2 документах из 5.
$$IDF = \log(5/2) \approx 0.91$$

Поскольку локальная частота (TF) всех слов в рамках короткого предложения одинакова, итоговый вес $TF \times IDF$ оказывается значительно выше у уникальных для корпуса лексем.

Рисунок 6 — Анализ значений TF-IDF и математическое обоснование разницы в весах на основе формулы IDF.

Задание 3. Модификация регулярных выражений под сохранение числовых данных

```
modified_corpus = corpus.copy()
modified_corpus[0] = "Сегодня в 2026 году была отличная погода, и мы пошли гулять в красивый парк!"

def pipeline_preprocess_with_digits(text):
    text = text.lower()
    text = re.sub(r'[^а-яё0-9\s]', '', text)
    doc = nlp(text)
    return [token.lemma_ for token in doc if token.lemma_ not in russian_stopwords and token.lemma_.strip()]

sample_text = modified_corpus[0]
cleaned_text = re.sub(r'[^а-яё0-9\s]', '', sample_text.lower())
doc_demo = nlp(cleaned_text)

for token in doc_demo:
    if token.text.strip():
        print(f"{token.text:<10} | {token.lemma_}")

print(pipeline_preprocess_with_digits(sample_text))
```

```
сегодня    | сегодня
в          | в
2026       | 2026
году       | год
была       | быть
отличная   | отличный
погода     | погода
и          | и
мы         | мы
пошли      | пойти
гулять     | гулять
в          | в
красивый   | красивый
парк       | парк
['сегодня', '2026', 'год', 'отличный', 'погода', 'пойти', 'гулять', 'красивый', 'парк']
```

Рисунок 7 — Модификация шаблона очистки регулярных выражений на r'[^а-яё0-9\s]' для совместного сохранения буквенных и цифровых данных в текстовом потоке.

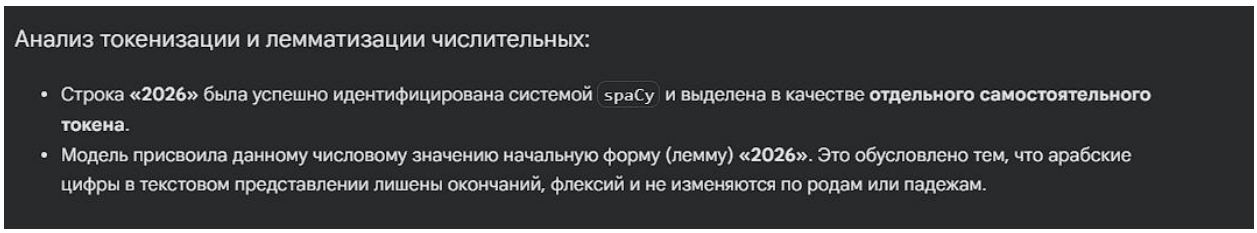


Рисунок 8 — Оценка результатов работы токенизатора модели spaCy при обработке строковых элементов, содержащих арабские цифры.

Вывод

В ходе выполнения лабораторной работы были освоены базовые этапы предварительного анализа естественного языка (NLP) на русском языке. На практике изучены механизмы очистки текста с помощью регулярных выражений, токенизации, фильтрации стоп-слов и лемматизации текстовых массивов с применением инструментов `spaCy` и `NLTK`. Было экспериментально доказано, что расширение исходного корпуса текстов напрямую увеличивает размерность признакового пространства модели TF-IDF.

- **Критически важно сохранять числовые данные:** В специализированных прикладных задачах NLP, таких как автоматический анализ кассовых чеков (суммы, даты), парсинг почтовых адресов (номера домов), распознавание номеров телефонов, артикулов товаров или извлечение временных меток.
- **Числа являются избыточным шумом:** При классификации литературных произведений по авторам, определении жанровой принадлежности статей или анализе эмоциональной тональности отзывов, где ключевую роль играет семантика слов, а точные числовые параметры создают ложные корреляции.

Ответы на контрольные вопросы

1. **Какова роль регулярных выражений (RegEx) при предварительной очистке текстового шума?**
   Очистка текста от внеязыкового мусора (HTML-тегов, знаков препинания, спецсимволов). Это нормализует текстовый поток перед токенизацией.
2. **К каким ошибкам в анализе может привести избыточное удаление спецсимволов и дефисов (например, в словах «e-mail» или «он-лайн»)?**
   Искажение смысла слов и их ошибочное дробление на изолированные бессмысленные буквенные токены (например, «e» и «mail»).
3. **Почему приведение текста к нижнему регистру (lowercase) является обязательным шагом перед токенизацией?**
   Для исключения дублирования одинаковых слов в словаре, написанных с разным регистром из-за позиции в предложении (например, «Погода» и «погода»).
4. **Как настроить регулярное выражение, чтобы оно сохраняло в тексте и буквы, и важные цифровые данные (например, даты или артикулы)?**
   Добавить цифровой диапазон `0-9` внутрь инвертированного класса исключения: `r'[^a-яё0-9\s]'`.
5. **В каких задачах обработки текста нельзя удалять знаки препинания и смайлики (emoji)?**
   В задачах Sentiment Analysis (анализ тональности), где знаки экспрессии (восклицательные знаки) и эмодзи напрямую несут эмоциональный заряд.
6. **В чем заключается главное различие между процессами лемматизации и стемминга?**
   Лемматизация определяет реальную начальную форму слова с учетом морфологии и контекста, а стемминг лишь грубо отсекает окончания до псевдоосновы.
7. **Почему для флективных языков (таких как русский) лемматизация эффективнее стемминга?**
   В русском языке слова имеют богатую систему окончаний. Стемминг склеит разные по смыслу основы, тогда как лемматизация сохранит точную семантику.
8. **Как алгоритм токенизации определяет границы слов при наличии сокращений (например, «т.д.», «и т.п.»)?**
   Опирается на встроенные словари исключений и шаблоны, предотвращающие деление токена по внутренней точке.
9. **По какому принципу слова формируют список стоп-слов в библиотеках NLTK или spaCy?**
   По принципу их высокой частотности в языке при крайне низкой самостоятельной информационной ценности (союзы, предлоги, частицы).
10. **Что произойдет с качеством анализа тональности отзывов, если удалить частицу «не» как стоп-слово?**
    Качество резко снизится, так как смысл полярных выражений изменится на противоположный («не отличный» станет «отличный»).